



Universidad Nacional de La Matanza

Algoritmos y Estructuras de Datos

Guía Git y GitHub



Objetivo

- Aprender el flujo básico de trabajo en un proyecto de programación en **C** utilizando **Code::Blocks**.
- Gestionar el código fuente con **Git** y colaborar a través de **GitHub**.
- Simular un equipo de **tres desarrolladores** que trabajan en paralelo y luego integran sus aportes.

Proyecto

Implementación de una **calculadora** en lenguaje C que permita las operaciones:

- Suma
- Resta
- Multiplicación
- División
- Potenciación
- Radicación

Roles de los Desarrolladores

1. **Desarrollador A:** Arquitecto principal. Crea el esqueleto del proyecto. Se encarga de la creación del repositorio en GitHub y la subida del template. Implementa las funciones suma y resta.
2. **Desarrollador B:** Implementa las funciones multiplicación y división.
3. **Desarrollador C:** Implementa las funciones potencialización y radicación.

Pasos del Proyecto

1. Preparar el entorno

1.1. Cada desarrollador:

- Instala Code::Blocks
- Instala Git
- Abre una cuenta en GitHub.

1.2. Configurar Git en cada equipo (solo la primera vez):



```
git config --global user.name "Tu Nombre"
```

```
git config --global user.email "tuemail@example.com"
```

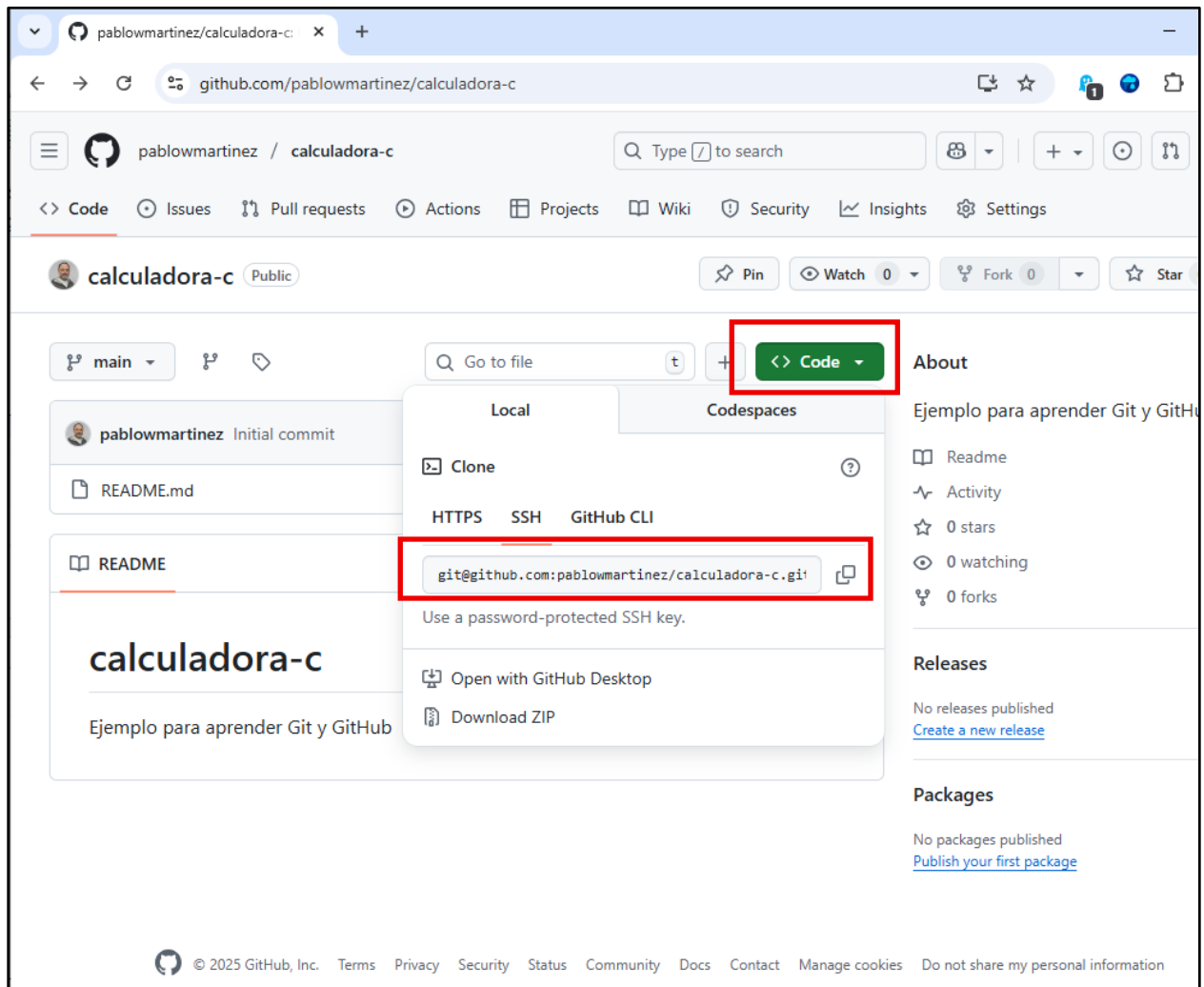
2. Crear el repositorio en GitHub

2.1. Un desarrollador (en este caso sería A) crea el repositorio en GitHub:

- Nombre: calculadora-c
- Opción: Repositorio público
- Inicializar con README.md

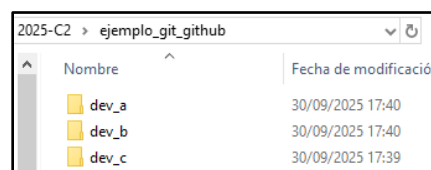
The screenshot shows the GitHub 'New repository' page. The browser address bar shows 'github.com/new'. The page title is 'New repository'. The main heading is 'Create a new repository'. Below this, there is a subheading 'Repositories contain a project's files and version history. Have a project elsewhere? [Import a repository.](#) Required fields are marked with an asterisk (*).'. The page is divided into two sections: '1 General' and '2 Configuration'. In the 'General' section, the 'Owner' is 'pablowmartinez' and the 'Repository name' is 'calculadora-c'. A green checkmark indicates 'calculadora-c is available.'. The 'Description' field contains 'Ejemplo para aprender Git y GitHub'. In the 'Configuration' section, the 'Choose visibility' dropdown is set to 'Public'. The 'Add README' checkbox is checked. The 'Add .gitignore' dropdown is set to 'No .gitignore'. The 'Add license' dropdown is set to 'No license'. A green 'Create repository' button is at the bottom right.

2.2. Copiar la URL del repositorio (HTTPS o SSH) desde el botón Code.

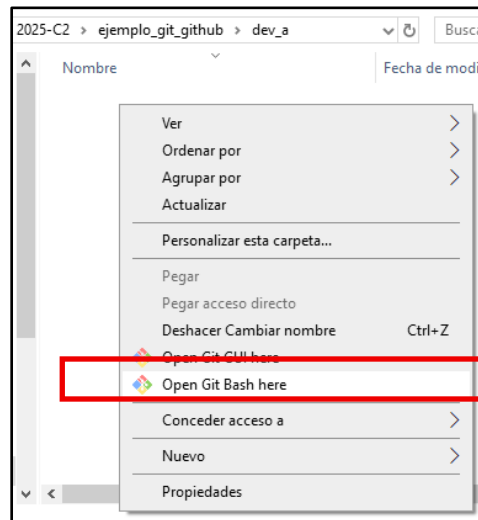


3. Clonar el repositorio

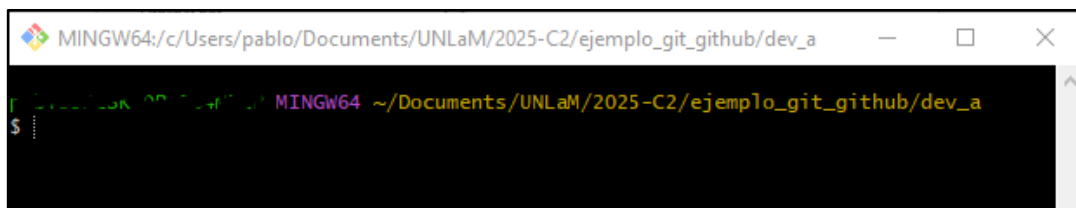
Cada desarrollador clona el repositorio en su equipo (utiliza la URL del repositorio copiada en el punto anterior. Abrir Git bash en la carpeta local sobre la que se va a trabajar (recordar que se van a simular tres desarrolladores y por ello, serán tres carpetas:



Ingresa en la carpeta y hacer click con el botón derecho del mouse para abrir el menú y activar Git Bash:

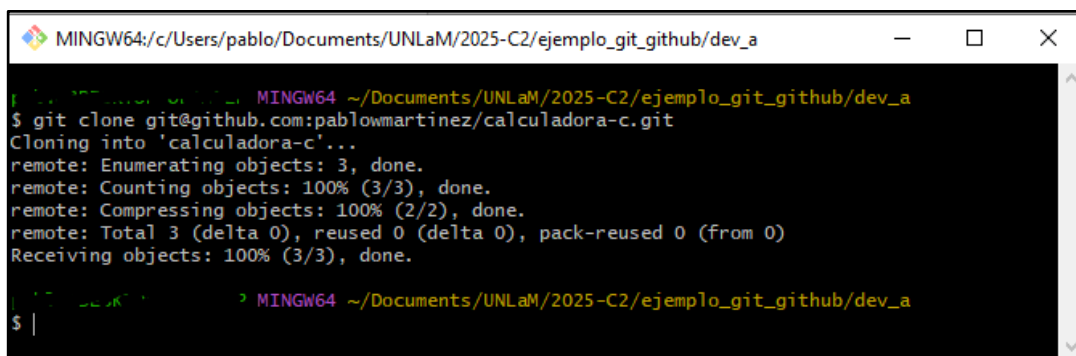


Se abrirá Git Bash:



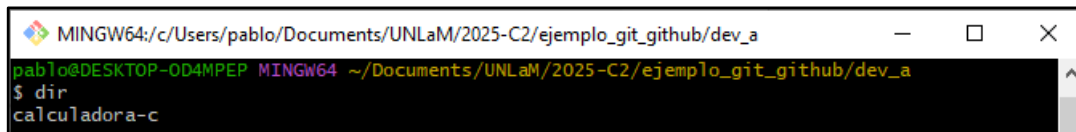
Clonar el repositorio con el comando:

```
git clone git@github.com:pablowmartinez/calculadora-c.git
```



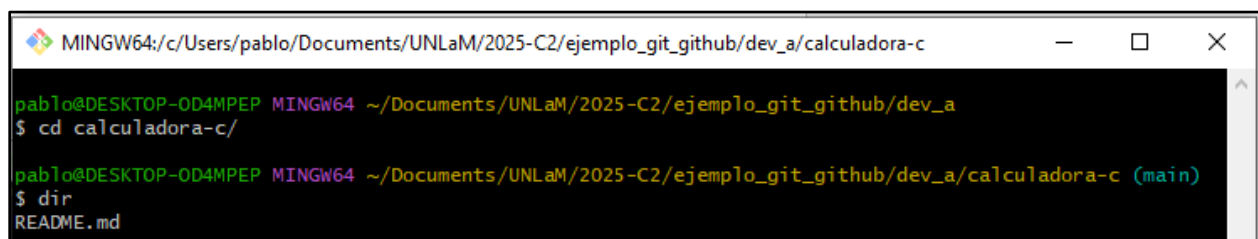


Una vez finalizado el clonado del repositorio verificar que se haya creado la carpeta “calculadora-c”:

A terminal window titled 'MINGW64; c:/Users/pablo/Documents/UNLaM/2025-C2/ejemplo_git_github/dev_a'. The prompt is 'pablo@DESKTOP-OD4MPEP MINGW64 ~/Documents/UNLaM/2025-C2/ejemplo_git_github/dev_a'. The user enters '\$ dir' and the output is 'calculadora-c'.

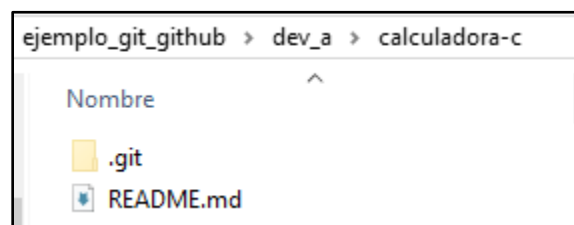
Ingresar en la carpeta y luego listar el contenido de esta con los comandos:

- `cd calculadora-c`
- `dir`

A terminal window titled 'MINGW64; c:/Users/pablo/Documents/UNLaM/2025-C2/ejemplo_git_github/dev_a/calculadora-c'. The prompt is 'pablo@DESKTOP-OD4MPEP MINGW64 ~/Documents/UNLaM/2025-C2/ejemplo_git_github/dev_a'. The user enters '\$ cd calculadora-c/'. The prompt changes to 'pablo@DESKTOP-OD4MPEP MINGW64 ~/Documents/UNLaM/2025-C2/ejemplo_git_github/dev_a/calculadora-c (main)'. The user enters '\$ dir' and the output is 'README.md'.

Se podrá ver dentro de la carpeta el archivo README.md que fue descargado desde GitHub.

Desde el explorador de archivos de Windows se puede ver el contenido de la carpeta:



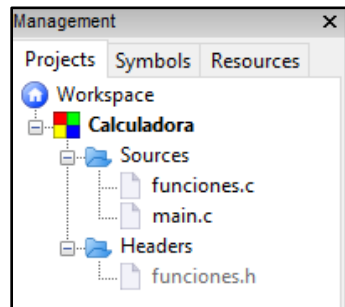
4. Crear la estructura inicial del proyecto

El Desarrollador A abre Code::Blocks y crea un nuevo proyecto “Calculadora” dentro de la carpeta del repositorio local y crea los archivos:

- `main.c`
- `funciones.h`
- `funciones.c`



Agrega la estructura mínima para que el proyecto compile y sirva de base a la producción de los demás desarrolladores del equipo:



El contenido de los archivos será:

```
main.c X
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include "funciones.h"
4
5  int main()
6  {
7      printf("Calculadora en C\n");
8      return 0;
9  }
10
```

```
funciones.h X
1  #ifndef FUNCIONES_H_INCLUDED
2  #define FUNCIONES_H_INCLUDED
3
4
5
6  #endif // FUNCIONES_H_INCLUDED
7
```

```
funciones.c X
1  #include "funciones.h"
2
3
```

Crear el archivo .gitignore:

```
.gitignore: Bloc de notas
Archivo Edición Formato Ver Ayuda
bin/
obj/
*.layout
*.depend
```



5. Subir la primera versión al repositorio:

Luego de crear el proyecto base vemos el estado del repositorio (utilizando el comando git status):

A screenshot of a terminal window with a black background and white text. The window title is 'MINGW64:/c/Users/pablo/Documents/UNLaM/2025-C2/ejemplo_git_github/dev_a/calculadora-c'. The terminal shows the command '\$ git status' and its output: 'On branch main', 'Your branch is up to date with 'origin/main'.', 'Untracked files:', '(use "git add <file>..." to include in what will be committed)', '.gitignore', 'Calculadora.cbp', 'funciones.c', 'funciones.h', 'main.c', and 'nothing added to commit but untracked files present (use "git add" to track)'.

```
MINGW64:/c/Users/pablo/Documents/UNLaM/2025-C2/ejemplo_git_github/dev_a/calculadora-c
pablo@DESKTOP-OD4MPEP MINGW64 ~/Documents/UNLaM/2025-C2/ejemplo_git_github/dev_a/calculadora-c (main)
$ git status
On branch main
Your branch is up to date with 'origin/main'.

Untracked files:
(use "git add <file>..." to include in what will be committed)
.gitignore
Calculadora.cbp
funciones.c
funciones.h
main.c

nothing added to commit but untracked files present (use "git add" to track)
```

Vemos que aparecen archivos que no se han incluido. Incluir los archivos en la zona de staging, luego hacer el commit y subir los cambios a GitHub con los siguientes comandos:

```
git add .
```

```
git commit -m "Estructura inicial del proyecto"
```

```
git push origin main
```

La siguiente figura muestra el resultado de la ejecución de los comandos:



UNLaM

Dto. Ingeniería e Investigaciones Tecnológicas

```
pablo@DESKTOP-OD4MPEP MINGW64 ~/Documents/UNLaM/2025-C2/ejemplo_git_github/dev_a/calculadora-c (main)
$ git add .

pablo@DESKTOP-OD4MPEP MINGW64 ~/Documents/UNLaM/2025-C2/ejemplo_git_github/dev_a/calculadora-c (main)
$ git status
On branch main
Your branch is up to date with 'origin/main'.

Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
    new file:   .gitignore
    new file:   Calculadora.cbp
    new file:   funciones.c
    new file:   funciones.h
    new file:   main.c

pablo@DESKTOP-OD4MPEP MINGW64 ~/Documents/UNLaM/2025-C2/ejemplo_git_github/dev_a/calculadora-c (main)
$ git commit -m "Estructura inicial del proyecto"
[main 2d395c3] Estructura inicial del proyecto
5 files changed, 64 insertions(+)
create mode 100644 .gitignore
create mode 100644 Calculadora.cbp
create mode 100644 funciones.c
create mode 100644 funciones.h
create mode 100644 main.c

pablo@DESKTOP-OD4MPEP MINGW64 ~/Documents/UNLaM/2025-C2/ejemplo_git_github/dev_a/calculadora-c (main)
$ git status
On branch main
Your branch is ahead of 'origin/main' by 1 commit.
  (use "git push" to publish your local commits)

nothing to commit, working tree clean

pablo@DESKTOP-OD4MPEP MINGW64 ~/Documents/UNLaM/2025-C2/ejemplo_git_github/dev_a/calculadora-c (main)
$ git log
commit 2d395c3ad34ec51c3490a65a9249993907ec5f91 (HEAD -> main)
Author: Pablo <pablowmartinez@hotmail.com>
Date:   Tue Sep 30 19:06:06 2025 +0200

    Estructura inicial del proyecto

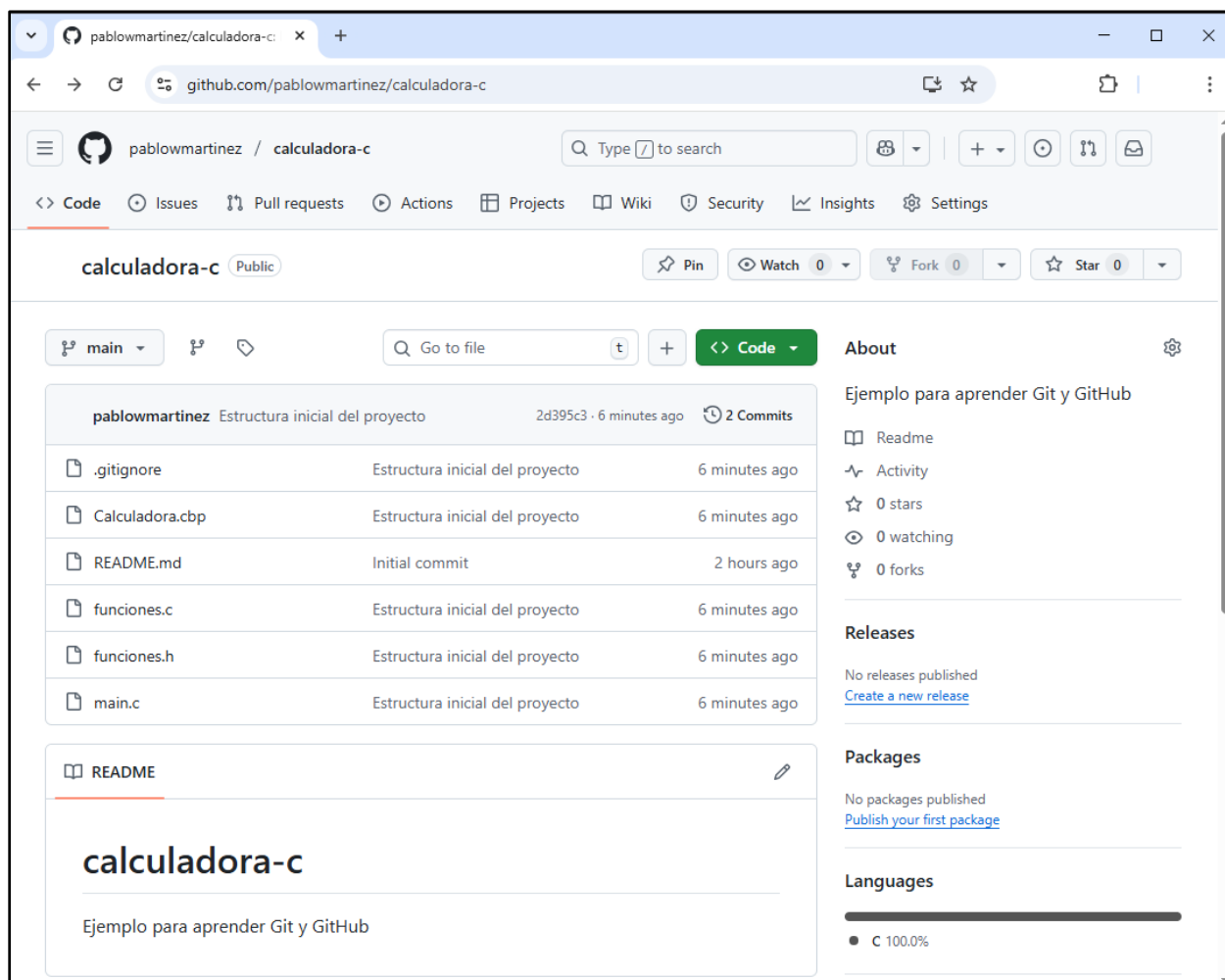
commit d731128d097cd8054ea69c112a3a5b146572cc46 (origin/main, origin/HEAD)
Author: Pablo Witold Martinez <pablowmartinez@hotmail.com>
Date:   Tue Sep 30 17:10:42 2025 +0200

    Initial commit

pablo@DESKTOP-OD4MPEP MINGW64 ~/Documents/UNLaM/2025-C2/ejemplo_git_github/dev_a/calculadora-c (main)
$ git push origin main
Enumerating objects: 8, done.
Counting objects: 100% (8/8), done.
Delta compression using up to 8 threads
Compressing objects: 100% (5/5), done.
Writing objects: 100% (7/7), 1.01 KiB | 206.00 KiB/s, done.
Total 7 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To github.com:pablowmartinez/calculadora-c.git
   d731128..2d395c3  main -> main
```



Al haber hecho el push a GitHub, en la web del repositorio se podrán ver los cambios que se han subido:



Los Desarrolladores B y C deben realizar el procedimiento del punto 3 para clonar el repositorio en sus equipos y comenzar con el desarrollo.

6. Crear ramas para cada desarrollador

Para trabajar sin pisarse, cada desarrollador creará su propia rama de desarrollo sobre el proyecto.

En el paso anterior (paso 5) el Arquitecto del proyecto (desarrollador A) creó un proyecto base, lo subió a GitHub y cada uno de los desarrolladores del equipo (desarrolladores B y C) clonaron



el repositorio en su equipo local. Hasta ahora el proyecto posee una sola rama (**main**) y dos commits, el inicial y el correspondiente a la estructura del proyecto. Esto se puede ver con el comando `git log`:

```
MINGW64 ~/Documents/UNLaM/2025-C2/ejemplo_git_github/dev_a/calculadora-c (main)
$ git log
commit 2d395c3ad34ec51c3490a65a9249993907ec5f91 (HEAD -> main, origin/main, origin/HEAD)
Author: Pablo <pablowmartinez@hotmail.com>
Date: Tue Sep 30 19:06:06 2025 +0200

    Estructura inicial del proyecto

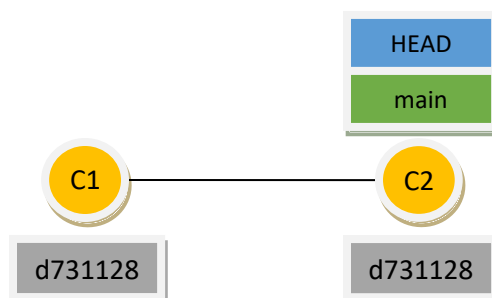
commit d731128d097cd8054ea69c112a3a5b146572cc46
Author: Pablo Witold Martinez <pablowmartinez@hotmail.com>
Date: Tue Sep 30 17:10:42 2025 +0200

    Initial commit
```

También se puede ver la versión resumida a una línea por commit con el comando `git log --oneline`:

```
MINGW64 ~/Documents/UNLaM/2025-C2/ejemplo_git_github/dev_a/calculadora-c (main)
$ git log --oneline
2d395c3 (HEAD -> main, origin/main, origin/HEAD) Estructura inicial del proyecto
d731128 Initial commit
```

En la siguiente figura podemos ver la representación de las ramas y commits que tenemos al momento en el repositorio. Los círculos representan los commits. Los rectángulos grises con el número hexadecimal representan los identificadores de los commits. El rectángulo verde indica la rama (en este caso **main**). Finalmente, el rectángulo azul representa el puntero (**HEAD**) que indica en qué commit nos encontramos (en este ejemplo estamos en **d731128**)





Lo que nos interesa a partir de este momento es que cada desarrollador genere su propia rama de desarrollo independiente de la rama principal (main), trabaje en ella y no interfiera con el resto del equipo de desarrollo.

Entonces, crear las ramas propias:

- Desarrollador A:
 - `git checkout -b dev-a`
- Desarrollador B:
 - `git checkout -b dev-b`
- Desarrollador C:
 - `git checkout -b dev-c`

Luego de ejecutar los comandos para el Desarrollador A veremos que se crea una nueva rama y se mueve a la nueva rama creada (en la figura se puede ver como pasa de la rama **main** a **dev-a**):

```
pablo@DESKTOP-OD4MPEP MINGW64 ~/Documents/UNLaM/2025-C2/ejemplo_git_github/dev_a/calculadora-c (main)
$ git checkout -b dev-a
Switched to a new branch 'dev-a'

pablo@DESKTOP-OD4MPEP MINGW64 ~/Documents/UNLaM/2025-C2/ejemplo_git_github/dev_a/calculadora-c (dev-a)
$ |
```

Repetir los comandos dentro de cada carpeta para simular el proceso para cada “desarrollador”.



7. Desarrollo del código

Cada miembro del equipo desarrolla su parte del proyecto asignada.

Desarrollador A:

```
main.c X
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include "funciones.h"
4
5  int main()
6  {
7      float a = 10;
8      float b = 5;
9      printf("Calculadora en C\n");
10     printf("%.2f + %.2f = %.2f\n", a, b, suma(a, b));
11     printf("%.2f - %.2f = %.2f\n", a, b, resta(a, b));
12     return 0;
13 }
14
```

```
funciones.h X
1  #ifndef FUNCIONES_H_INCLUDED
2  #define FUNCIONES_H_INCLUDED
3
4  float suma(float a, float b);
5  float resta(float a, float b);
6
7  #endif // FUNCIONES_H_INCLUDED
8
```

```
funciones.c X
1  #include "funciones.h"
2
3  float suma(float a, float b)
4  {
5      return a + b;
6  }
7
8  float resta(float a, float b)
9  {
10     return a - b;
11 }
12
```



UNLaM

Dto. Ingeniería e Investigaciones Tecnológicas

Desarrollador B:

```
main.c X
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include "funciones.h"
4
5  int main()
6  {
7      float a = 10;
8      float b = 5;
9      printf("Calculadora en C\n");
10     printf("%.2f x %.2f = %.2f\n", a, b, multiplicar(a, b));
11     printf("%.2f / %.2f = %.2f\n", a, b, dividir(a, b));
12     return 0;
13 }
14
```

```
funciones.h X
1  #ifndef FUNCIONES_H_INCLUDED
2  #define FUNCIONES_H_INCLUDED
3
4  float multiplicar(float a, float b);
5  float dividir(float a, float b);
6
7  #endif // FUNCIONES_H_INCLUDED
8
```

```
funciones.c X
1  #include "funciones.h"
2
3  float multiplicar(float a, float b)
4  {
5      return a * b;
6  }
7
8  float dividir(float a, float b)
9  {
10     if (b == 0)
11     {
12         return 0; // Manejo simple de error
13     }
14     return a / b;
15 }
16
```



UNLaM

Dto. Ingeniería e Investigaciones Tecnológicas

Desarrollador C:

```
main.c X
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include "funciones.h"
4
5  int main()
6  {
7      float a = 4;
8      float b = 2;
9      printf("Calculadora en C\n");
10     printf("%.2f elevado a %.2f = %.2f\n", a, b, potencia(a, b));
11     printf("Raiz %.2f de %.2f = %.2f\n", b, a, raiz(a, b));
12     return 0;
13 }
14
```

```
funciones.h X
1  #ifndef FUNCIONES_H_INCLUDED
2  #define FUNCIONES_H_INCLUDED
3
4  #include <math.h>
5
6  float potencia(float base, float exponente);
7  float raiz(float numero, float indice);
8
9  #endif // FUNCIONES_H_INCLUDED
10
```

```
funciones.c X
1  #include "funciones.h"
2
3  float potencia(float base, float exponente)
4  {
5      return pow(base, exponente);
6  }
7
8  float raiz(float numero, float indice)
9  {
10     return pow(numero, 1.0 / indice);
11 }
12
```



8. Subir los cambios a GitHub

Cada desarrollador sube su **rama con los cambios** a GitHub. Los comandos para el desarrollador A serían:

```
git add .
```

```
git commit -m "Implementación de las operaciones suma y resta"
```

```
git push origin dev-a
```

Notar que en **git push** se sube la rama **dev-a**. Luego de hacer el push, ir a GitHub para ver que es lo que ha sucedido. Luego, repetir los pasos para los desarrolladores B y C.

9. Revisar y hacer Pull Request

Cada vez que se haga un push sobre GitHub será necesario crear un **Pull Request (PR)** Esto es necesario porque se desea integrar los cambios que un desarrollador ha hecho al proyecto principal. El PR es una solicitud para integrar los cambios. Entonces:

- **¿Cuándo se hace?**

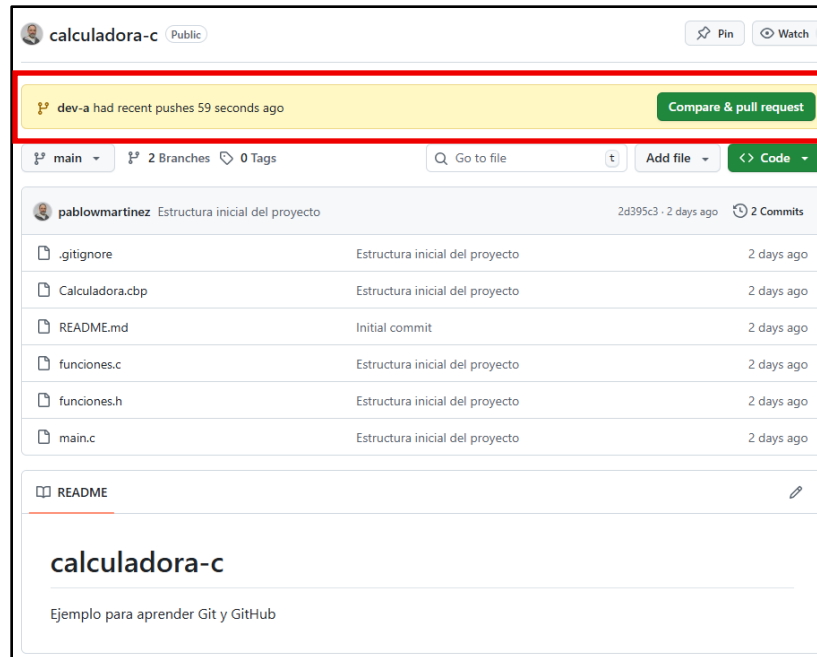
Cuando terminaste de trabajar en una tarea (como arreglar un error o añadir una función) en tu propia copia del proyecto.

- **¿Por qué se hace?**

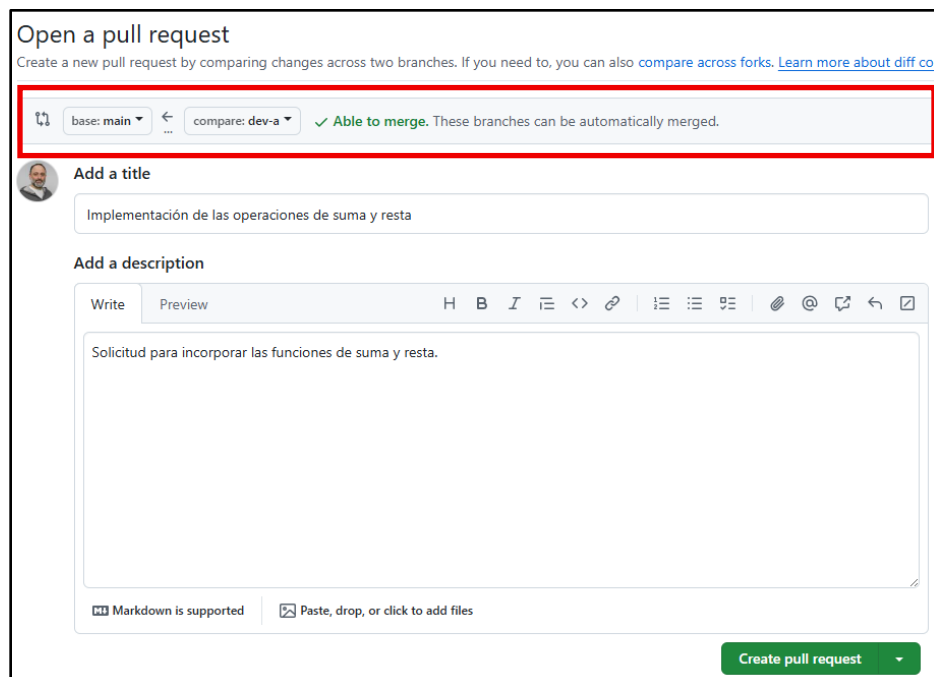
Para **revisar y discutir** el código antes de mezclarlo. Es como decir: *"Equipo, hice estos cambios. ¿Los pueden revisar y, si están bien, los agregan al proyecto?"*.

Resumiendo: Es el proceso formal para proponer, revisar y aprobar mejoras en un proyecto colaborativo. Recuerden que no queremos romper lo que ya está funcionando con los cambios.

Volviendo a nuestro proyecto luego de que el Desarrollador A ha realizado el Push se podrá ver en GitHub que aparece un Pull Request disponible para hacer:



Luego de darle al botón “Compare & pull request” aparecerá el formulario para crear el PR. Noten que en el recuadro rojo de la siguiente figura se ve que los cambios de la rama **dev-a** serán aplicadas a la rama **main** del proyecto en GitHub:





Antes de crear el PR con el botón “Create pull request” es interesante observar cuales son los cambios detectados:

Commits on Oct 2, 2025

Implementacion de las operaciones de suma y resta
pablowmartinez committed 2 days ago

Showing 3 changed files with 15 additions and 1 deletion.

funciones.c

```
@@ -1,2 +1,11 @@
1 1 #include "funciones.h"
2 2
3 + float suma(float a, float b)
4 + {
5 +     return a + b;
6 + }
7 +
8 + float resta(float a, float b)
9 + {
10 +     return a - b;
11 + }
```

funciones.h

```
@@ -1,6 +1,7 @@
1 1 #ifndef FUNCIONES_H_INCLUDED
2 2 #define FUNCIONES_H_INCLUDED
3 3
4 -
4 + float suma(float a, float b);
5 + float resta(float a, float b);
5 6
6 7 #endif // FUNCIONES_H_INCLUDED
```

main.c

```
@@ -4,6 +4,10 @@
4 4
5 5 int main()
6 6 {
7 +     float a = 10;
8 +     float b = 5;
7 9 printf("Calculadora en C\n");
10 + printf("%.2f + %.2f = %.2f\n", a, b, suma(a, b));
11 + printf("%.2f - %.2f = %.2f\n", a, b, resta(a, b));
8 12 return 0;
9 13 }
```

Noten la coincidencia entre el ID del commit en GitHub y el ID del commit local:

```
MINGW64 ~/Documents/UNLaM/2025-C2/ejemplo_git_github/dev_a/calculadora-c (dev-a)
$ git log --oneline
9470408 (HEAD -> dev-a, origin/dev-a) Implementacion de las operaciones de suma y resta
2d395c3 (origin/main, origin/HEAD, main) Estructura inicial del proyecto
d731128 Initial commit
```

Luego de crear el PR, se verifica que no existan conflictos. Si todo va bien GitHub lo indicará y permitirá hacer el merge (botón “Merge pull request”):



UNLaM

Dto. Ingeniería e Investigaciones Tecnológicas

Implementación de las operaciones de suma y resta #1

Open pablowmartinez wants to merge 1 commit into `main` from `dev-a`

Conversation 0

Commits 1

Checks 0

Files changed 3



pablowmartinez commented 2 minutes ago

Owner ...

Solicitud para incorporar las funciones de suma y resta.



Implementacion de las operaciones de suma y resta

9470488



No conflicts with base branch

Merging can be performed automatically.

Merge pull request

You can also merge this with the command line. [View command line instructions.](#)



Add a comment

Write

Preview

H

B

I

≡

<>

Add your comment here...

Markdown is supported

Paste, drop, or click to add files

Close with comment

Comment

Confirmar el merge (botón “Confirm merge”):



UNLaM

Dto. Ingeniería e Investigaciones Tecnológicas

Implementación de las operaciones de suma y resta #1

pablowmartinez wants to merge 1 commit into `main` from `dev-a`

Conversation 0



Commits 1



Checks 0



Files changed 3



pablowmartinez commented 5 minutes ago

Owner ...

Solicitud para incorporar las funciones de suma y resta.



Implementacion de las operaciones de suma y resta

9470408



Commit message

Merge pull request #1 from pablowmartinez/dev-a

Extended description

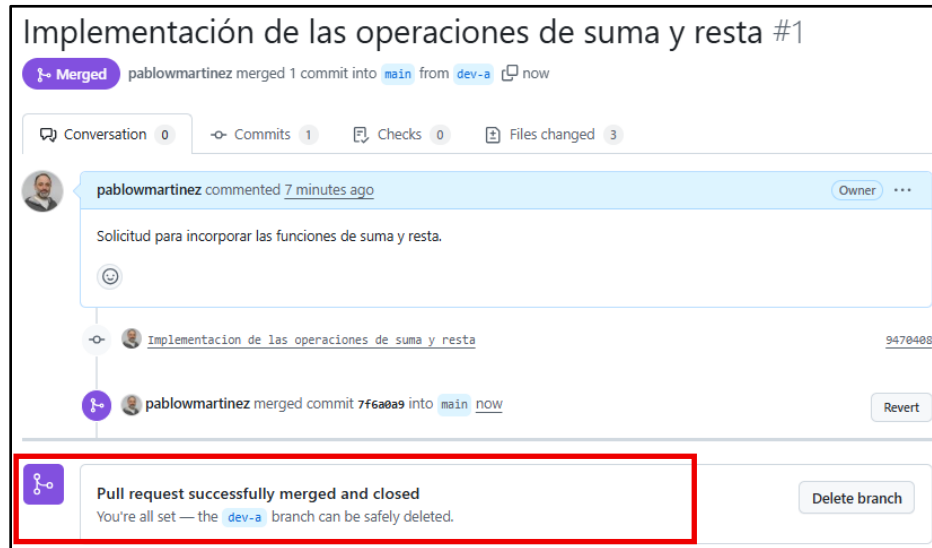
Implementación de las operaciones de suma y resta

This commit will be authored by pablowmartinez@hotmail.com.

Confirm merge

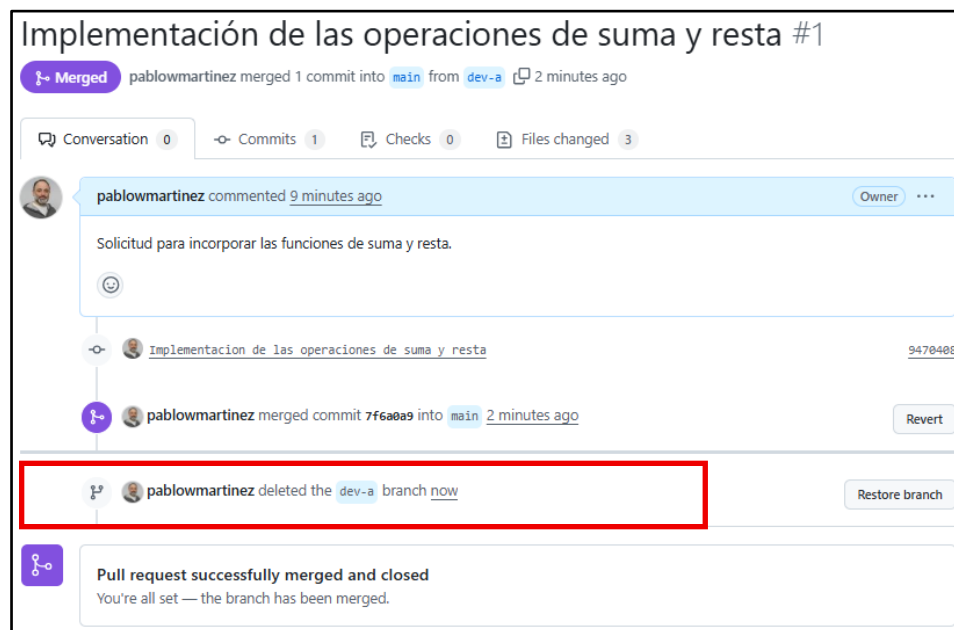
Cancel

Una vez confirmado el merge, GitHub lo hará efectivo y lo indicará de la siguiente manera:



Noten el cambio de  a .

Una opción que GitHub provee es la eliminación la rama (branch) de la que provienen los cambios (dev-a). Su eliminación contribuye a la limpieza y claridad del flujo de trabajo:

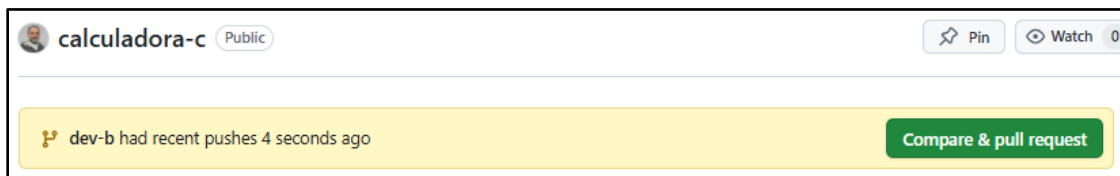




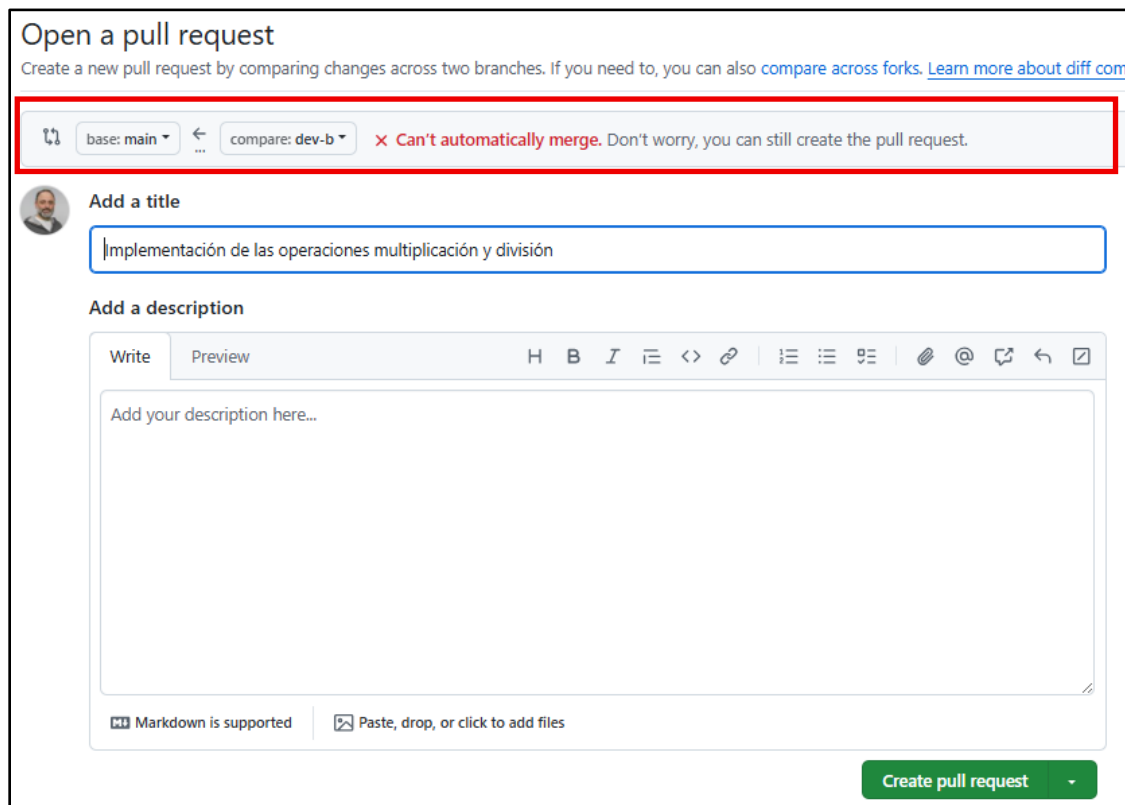
Hasta aquí hemos integrado los cambios introducidos por el Desarrollador A. Veamos que sucede con B y C:

Desarrollador B:

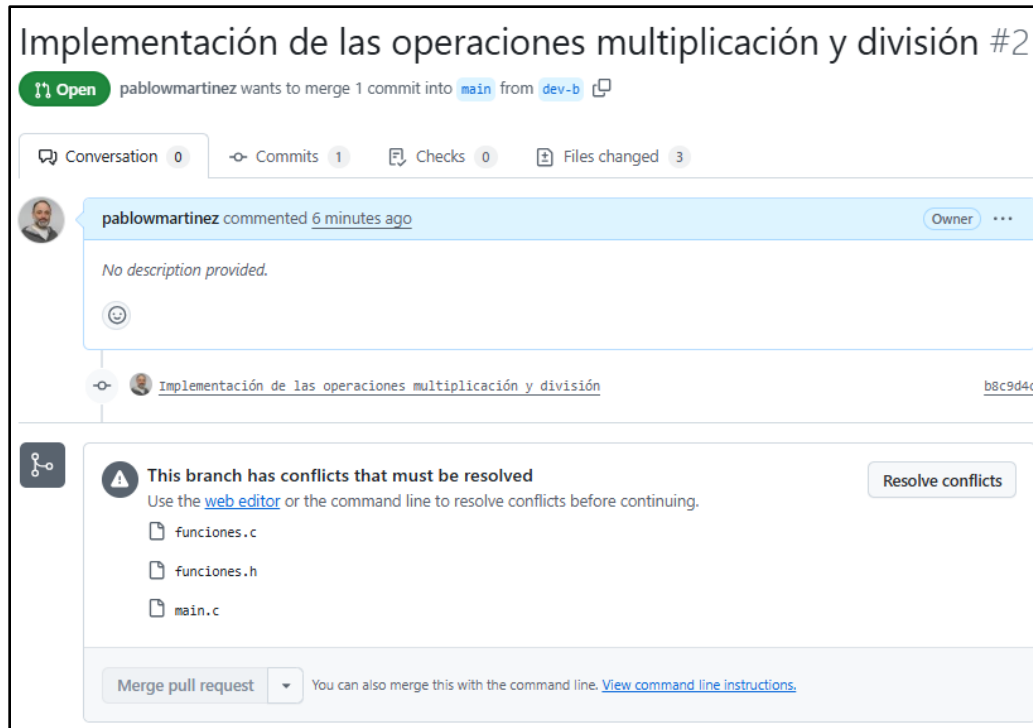
Luego de subir los cambios del repositorio local a GitHub, GitHub detecta el push:



Al realizar el “Compare & pull request” sucede lo siguiente:



GitHub nos está indicando que algo está sucediendo y no va a ser posible realizar el merge directamente. Estamos ante un **conflicto** y es una situación que sucede a menudo y es necesario aprender a resolver. Una vez que se cree el PR con el botón “Create pull request” veremos en **dónde** se presentan los conflictos:



Se puede ver que archivos se dan los conflictos. Ahora es momento de resolverlos. Para hacerlo hay que hacer click en el botón “resolve conflicts”:



El editor web de GitHub muestra los archivos y las diferencias a resolver. Veamos en detalle que sucede con los tres archivos:



UNLaM

Dto. Ingeniería e Investigaciones Tecnológicas

funciones.c:

```

funciones.c

1  #include "funciones.h"
2
3  <<<<<< dev-b (Current change)
4  float multiplicar(float a, float b)
5  {
6      return a * b;
7  }
8
9  float dividir(float a, float b)
10 {
11     if (b == 0)
12     {
13         return 0; // Manejo simple de error
14     }
15     return a / b;
16 }
17 =====
18 float suma(float a, float b)
19 {
20     return a + b;
21 }
22
23 float resta(float a, float b)
24 {
25     return a - b;
26 }
27 >>>>>> main (Incoming change)
28 {
29     // ...
30 }

```

Lo que hay en el repositorio (corresponden a los cambios que introdujo el desarrollador A):

```

funciones.c X
1  #include "funciones.h"
2
3  float suma(float a, float b)
4  {
5      return a + b;
6  }
7
8  float resta(float a, float b)
9  {
10     return a - b;
11 }
12

```

Lo que pretende introducir el desarrollador B:

```

funciones.c X
1  #include "funciones.h"
2
3  float multiplicar(float a, float b)
4  {
5      return a * b;
6  }
7
8  float dividir(float a, float b)
9  {
10     if (b == 0)
11     {
12         return 0; // Manejo simple de error
13     }
14     return a / b;
15 }
16

```




Lo que está sucediendo es que el archivo funciones.c alojado en el repositorio de GitHub difiere del archivo sibido por el desarrollador B y GitHub no puede resolver por si solo como realizar el merge correcto. Por ese motivo el equipo de trabajo debe resolver el conflicto.

Para ello se deberán editar y corregir todos los archivos que presenten conflictos.

Corregimos las diferencias y marcamos el conflicto como resuelto dándole al botón “Mark as resolved”:

This screenshot shows the GitHub web interface for resolving a conflict in the file 'funciones.c'. The title of the page is 'Implementación de las operaciones multiplicació... #2'. Below the title, it says 'Resolving conflicts between dev-b and main and committing changes → dev-b'. On the left, there is a sidebar with '3 conflicting files' listed: 'funciones.c', 'funciones.h', and 'main.c'. The main area shows the code for 'funciones.c' with line numbers 1 through 26. The code includes headers, function declarations for 'multiplicar', 'dividir', 'suma', and 'resta', and their corresponding implementations. A red '1 conflict' indicator is visible at the top right of the code editor. A 'Mark as resolved' button is located in the top right corner of the interface.

Archivo una vez resuelto el conflicto:

This screenshot shows the same GitHub web interface as the previous one, but now the conflict has been resolved. The title remains 'Implementación de las operaciones multiplicació... #2'. The sidebar still shows '3 conflicting files', but the 'funciones.c' file now has a green checkmark icon next to it, indicating it is resolved. The main area shows the same code for 'funciones.c' as before. The '1 conflict' indicator is gone, and a '✓ Resolved' status is shown in the top right corner of the code editor.



UNLaM

Dto. Ingeniería e Investigaciones Tecnológicas

funciones.h:

Implementación de las operaciones multiplicació... #2

Resolving conflicts between `dev-b` and `main` and committing changes → `dev-b`

3 conflicting files

funciones.h

1 conflict

Prev ^ Next v

Mark as resolved

funciones.c

funciones.h

main.c

```
1 #ifndef FUNCIONES_H_INCLUDED
2 #define FUNCIONES_H_INCLUDED
3
4 Accept current change | Accept incoming change | Accept both changes
5 <<<<<< dev-b (Current change)
6 float multiplicar(float a, float b);
7 float dividir(float a, float b);
8 =====
9 float suma(float a, float b);
10 float resta(float a, float b);
11 >>>>>> main (Incoming change)
12
13 #endif // FUNCIONES_H_INCLUDED
```

Implementación de las operaciones multiplicació... #2

Resolving conflicts between `dev-b` and `main` and committing changes → `dev-b`

3 conflicting files

funciones.h

✓ Resolved

funciones.c

funciones.h

main.c

```
1 #ifndef FUNCIONES_H_INCLUDED
2 #define FUNCIONES_H_INCLUDED
3
4 float multiplicar(float a, float b);
5 float dividir(float a, float b);
6 float suma(float a, float b);
7 float resta(float a, float b);
8
9 #endif // FUNCIONES_H_INCLUDED
10
```

main.c:

Implementación de las operaciones multiplicació... #2

Resolving conflicts between `dev-b` and `main` and committing changes → `dev-b`

3 conflicting files

File	Status
funciones.c	✓
funciones.h	✓
main.c	✗

main.c 1 conflict

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include "funciones.h"
4
5  int main()
6  {
7      float a = 10;
8      float b = 5;
9      printf("Calculadora en C\n");
10
11     <<<<<< dev-b (Current change)
12     printf("%.2f x %.2f = %.2f\n", a, b, multiplicar(a, b));
13     printf("%.2f / %.2f = %.2f\n", a, b, dividir(a, b));
14     =====
15     printf("%.2f + %.2f = %.2f\n", a, b, suma(a, b));
16     printf("%.2f - %.2f = %.2f\n", a, b, resta(a, b));
17     >>>>>> main (Incoming change)
18     return 0;
19 }
```

Accept current change | Accept incoming change | Accept both changes

Mark as resolved

Implementación de las operaciones multiplicació... #2

Resolving conflicts between `dev-b` and `main` and committing changes → `dev-b`

Commit merge

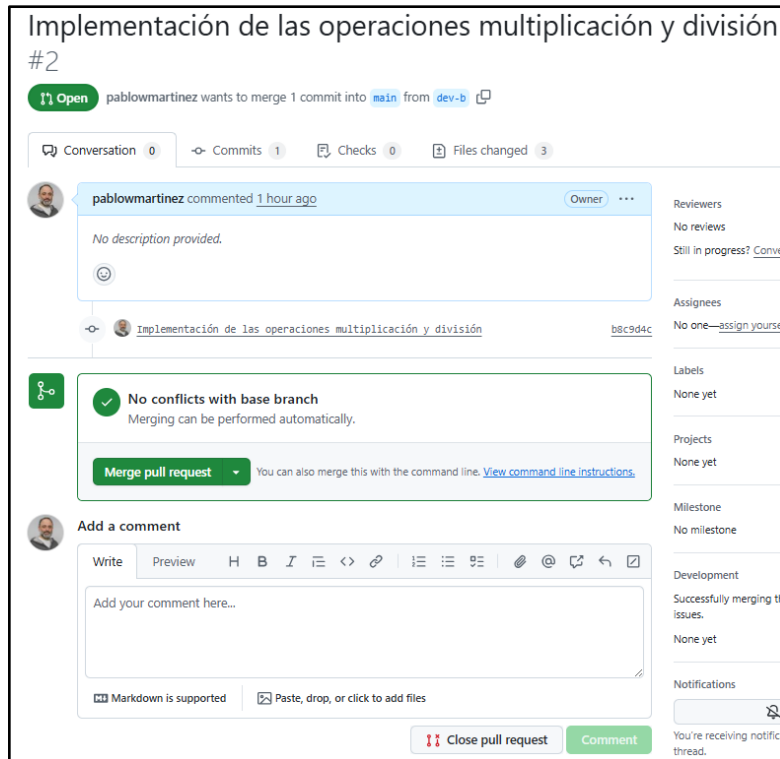
3 conflicting files

File	Status
funciones.c	✓
funciones.h	✓
main.c	✓

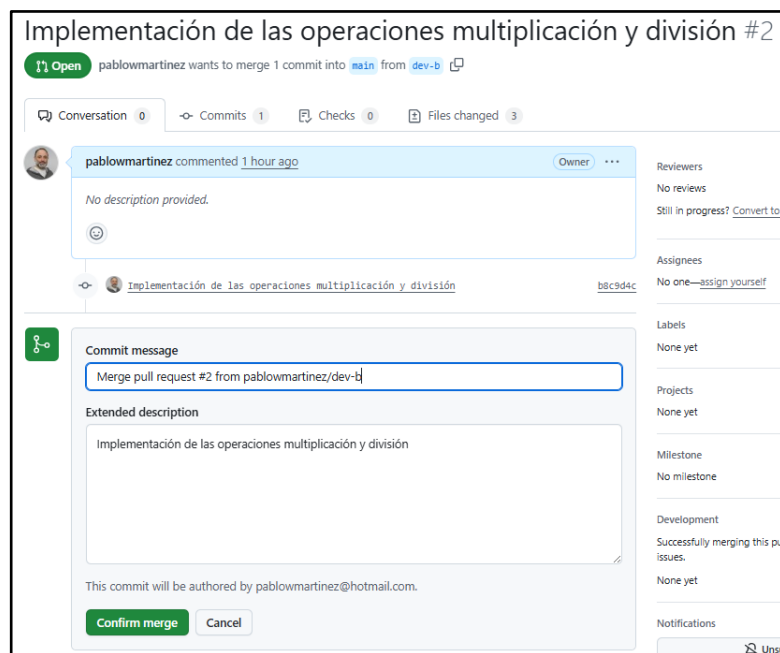
main.c ✓ Resolved

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include "funciones.h"
4
5  int main()
6  {
7      float a = 10;
8      float b = 5;
9      printf("Calculadora en C\n");
10     printf("%.2f + %.2f = %.2f\n", a, b, suma(a, b));
11     printf("%.2f - %.2f = %.2f\n", a, b, resta(a, b));
12     printf("%.2f x %.2f = %.2f\n", a, b, multiplicar(a, b));
13     printf("%.2f / %.2f = %.2f\n", a, b, dividir(a, b));
14     return 0;
15 }
16
```

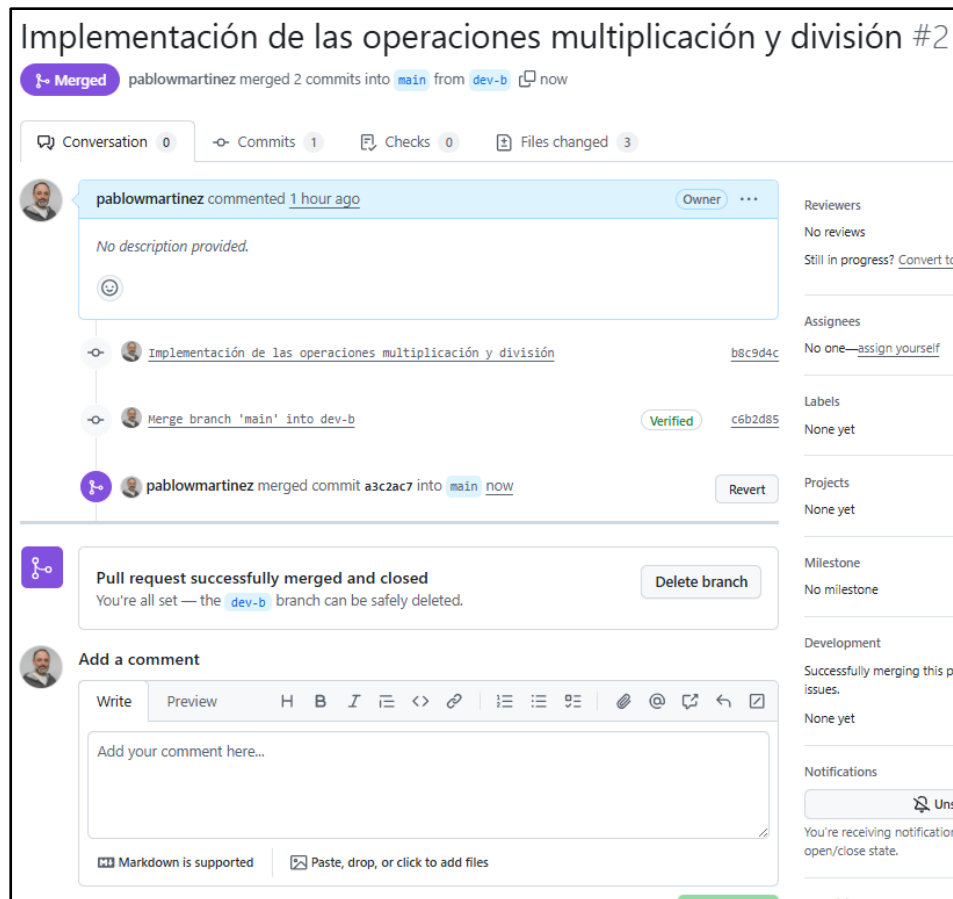
Una vez resueltos los conflictos podemos hacer el commit del merge con el botón "Commit merge". Podemos ver que ya es posible hacer el merge del PR:



Darle al botón y proseguir.



Confirmar el merge.



Merge realizado. Eliminar la rama y PR resuelto. YA se han incorporado los cambios del Desarrollador B sobre la rama main del proyecto.

Repetir el proceso con los cambios del desarrollador C (queda como tarea para Ustedes).

Esta fase es muy importante porque es la que integra y resuelve los cambios. Resumimos los pasos:

- En GitHub, cada desarrollador crea un **Pull Request** desde su rama hacia main.
- Se revisa el código (revisión cruzada entre compañeros).
- Se resuelven conflictos si los hubiera.
- Se aprueba y se hace el **merge**.

10. Probar el proyecto integrado.

- Para probar el proyecto, todos los desarrolladores actualizan su copia local.
 - `git checkout main`
 - `git pull origin main`
- Compilar en Code::Blocks y ejecutar.
- Verificar el correcto funcionamiento de todas las funciones integradas.

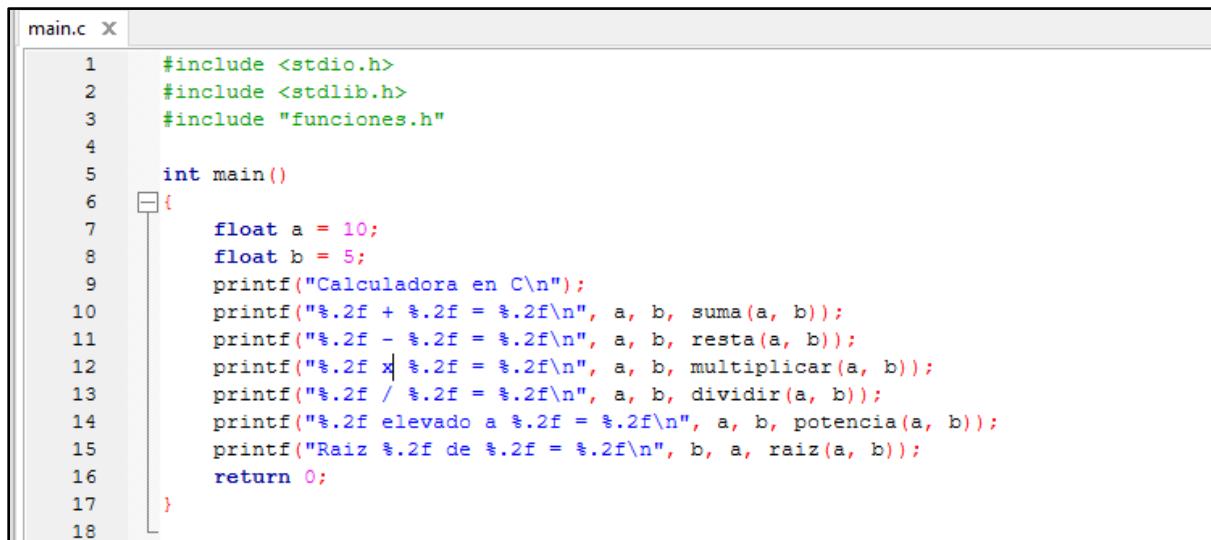
Probamos para el Desarrollador A:

Actualizar la copia local:

```
MINGW64 ~/Documents/UNLaM/2025-C2/ejemplo_git_github/dev_a/calculadora-c (dev-a)
$ git checkout main
Switched to branch 'main'
Your branch is up to date with 'origin/main'.

MINGW64 ~/Documents/UNLaM/2025-C2/ejemplo_git_github/dev_a/calculadora-c (main)
$ git pull origin main
remote: Enumerating objects: 31, done.
remote: Counting objects: 100% (31/31), done.
remote: Compressing objects: 100% (23/23), done.
remote: Total 23 (delta 9), reused 4 (delta 0), pack-reused 0 (from 0)
Unpacking objects: 100% (23/23), 6.40 KiB | 53.00 KiB/s, done.
From github.com:pablowmartinez/calculadora-c
* branch      main      -> FETCH_HEAD
   2d395c3..04e9724  main    -> origin/main
Updating 2d395c3..04e9724
Fast-forward
 funciones.c | 33 +++++
 funciones.h |  7 +++++
 main.c      |  8 +++++
 3 files changed, 48 insertions(+)
```

Vemos que los archivos en el repositorio local incluyen todos los cambios:



```
main.c X
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include "funciones.h"
4
5  int main()
6  {
7      float a = 10;
8      float b = 5;
9      printf("Calculadora en C\n");
10     printf("%.2f + %.2f = %.2f\n", a, b, suma(a, b));
11     printf("%.2f - %.2f = %.2f\n", a, b, resta(a, b));
12     printf("%.2f x %.2f = %.2f\n", a, b, multiplicar(a, b));
13     printf("%.2f / %.2f = %.2f\n", a, b, dividir(a, b));
14     printf("%.2f elevado a %.2f = %.2f\n", a, b, potencia(a, b));
15     printf("Raiz %.2f de %.2f = %.2f\n", b, a, raiz(a, b));
16     return 0;
17 }
18
```



UNLaM

Dto. Ingeniería e Investigaciones Tecnológicas

```
funciones.h X
1  #ifndef FUNCIONES_H_INCLUDED
2  #define FUNCIONES_H_INCLUDED
3
4  #include <math.h>
5
6  float potencia(float base, float exponente);
7  float raiz(float numero, float indice);
8  float multiplicar(float a, float b);
9  float dividir(float a, float b);
10 float suma(float a, float b);
11 float resta(float a, float b);
12
13 #endif // FUNCIONES_H_INCLUDED
14

funciones.c X
1  #include "funciones.h"
2
3  float potencia(float base, float exponente)
4  {
5      return pow(base, exponente);
6  }
7
8  float raiz(float numero, float indice)
9  {
10     return pow(numero, 1.0 / indice);
11 }
12
13 float multiplicar(float a, float b)
14 {
15     return a * b;
16 }
17
18 float dividir(float a, float b)
19 {
20     if (b == 0)
21     {
22         return 0; // Manejo simple de error
23     }
24     return a / b;
25 }
26
27 float suma(float a, float b)
28 {
29     return a + b;
30 }
31
32 float resta(float a, float b)
33 {
34     return a - b;
35 }
36
```



Proyecto compilado y en ejecución:

A screenshot of a Windows command prompt window. The title bar shows the file path: 'ejemplo_git_github\dev_a\calculadora-c\bin\Debug\Calculadora.exe'. The window content shows the output of a C program named 'Calculadora en C'. The output displays several arithmetic operations: '10.00 + 5.00 = 15.00', '10.00 - 5.00 = 5.00', '10.00 x 5.00 = 50.00', '10.00 / 5.00 = 2.00', '10.00 elevado a 5.00 = 100000.00', and 'Raiz 5.00 de 10.00 = 1.58'. At the bottom, it shows 'Process returned 0 (0x0) execution time : 7.490 s' and 'Press any key to continue.'.

```
ejemplo_git_github\dev_a\calculadora-c\bin\Debug\Calculadora.exe
Calculadora en C
10.00 + 5.00 = 15.00
10.00 - 5.00 = 5.00
10.00 x 5.00 = 50.00
10.00 / 5.00 = 2.00
10.00 elevado a 5.00 = 100000.00
Raiz 5.00 de 10.00 = 1.58

Process returned 0 (0x0)   execution time : 7.490 s
Press any key to continue.
```

Ciclo de trabajo en equipo (resumen)

Este es el resumen de pasos correspondientes al ciclo de trabajo:

1. **Clonar** repositorio.
2. **Crear rama** para cada tarea.
3. **Desarrollar** y hacer commits locales.
4. **Subir rama** (push).
5. **Crear Pull Request** en GitHub.
6. **Revisar, aprobar y mergear**.
7. **Actualizar main** en local.

Una vez iniciado el proyecto, no se clona el repositorio, se itera entre los puntos 2 y 7. En caso de que un nuevo desarrollador se integrara al equipo iniciaría desde el paso 1.

Comandos útiles

Veamos algunos comandos o grupos de comandos que pueden resultar útiles (tener en cuenta que Git tiene más de 140 comandos).

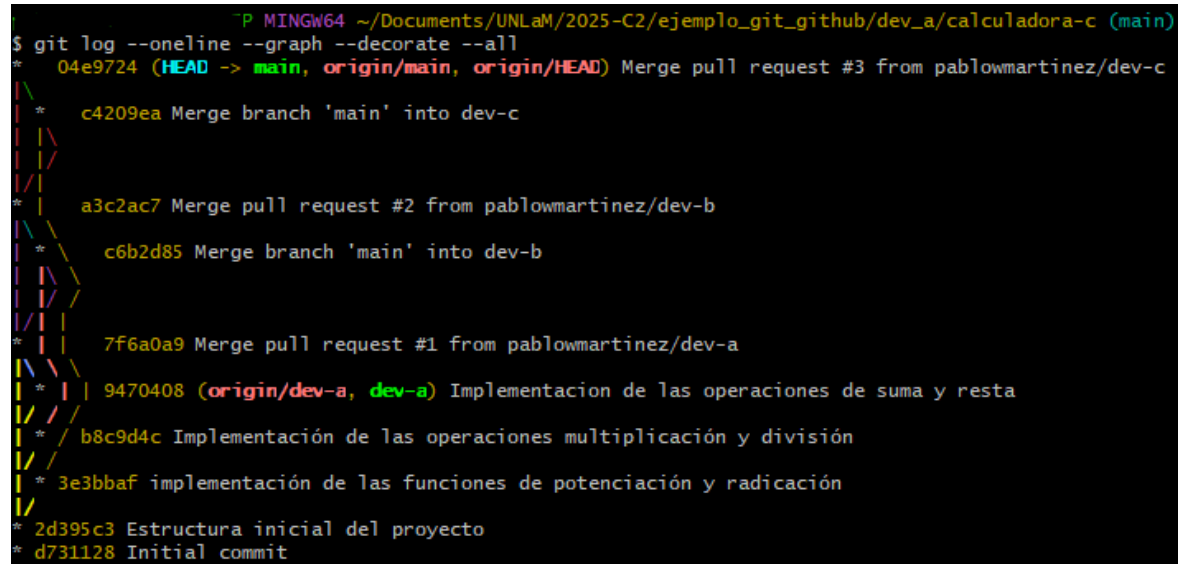
1. Ver las ramas del proyecto

A screenshot of a Windows command prompt window. The title bar shows the file path: 'MINGW64 ~/Documents/UNLaM/2025-C2/ejemplo_git_github/dev_a/calculadora-c (main)'. The window content shows the output of the 'git branch -l' command, which lists the local branches: 'dev-a' and 'main'. The 'main' branch is highlighted with an asterisk (*).

```
MINGW64 ~/Documents/UNLaM/2025-C2/ejemplo_git_github/dev_a/calculadora-c (main)
$ git branch -l
dev-a
* main
```


2. Ver el grafo de las ramas del proyecto

```
git log --oneline --graph --decorate --all
```



3. Regresar a un commit anterior

Supongamos que estamos trabajando Git y GitHub. Ya hemos hecho varios commits y push. Nos damos cuenta de que hemos cometido un error y queremos regresar a un commit anterior. Qué es lo que debemos hacer? Existen tres escenarios distintos y en función de lo que queramos hacer debemos situarnos en uno u otro. Veamos cuales son esos escenarios:

ESCENARIO 1: Queremos “**ver**” o “**probar**” un commit anterior (**sin borrar el trabajo actual**).

Comandos (los dos hacen lo mismo):

- `git checkout <commit_id>`
- `git switch -detach <commit_id>`

Esto lleva al commit en modo **detached HEAD** (no se está en ninguna rama). Es posible revisar o probar el estado de ese commit. Luego, para regresar al commit inicial:

- `git switch main`



UNLaM

Dto. Ingeniería e Investigaciones Tecnológicas

Tarea: investigar que implica el modo detached HEAD. Es un concepto importante para entender lo que pasa al moverse entre commits.

Ejemplo: iremos del commit final del proyecto (id = 04e9724) al commit con la estructura inicial del proyecto (id = 2d395c3) y observaremos que sucede con los archivos del proyecto de la calculadora.

Este es el log de commits realizados sobre el proyecto. Observen que estamos en la rama main:

```
MINGW64 ~/Documents/UNLaM/2025-C2/ejemplo_git_github/dev_c/calculadora-c (main)
$ git log --oneline
04e9724 (HEAD -> main, origin/main, origin/HEAD) Merge pull request #3 from pablowmartinez/dev-c
c4209ea Merge branch 'main' into dev-c
3e3bbaf (origin/dev-c, dev-c) implementación de las funciones de potenciación y radicación
a3c2ac7 Merge pull request #2 from pablowmartinez/dev-b
c6b2d85 Merge branch 'main' into dev-b
b8c9d4c Implementación de las operaciones multiplicación y división
7f6a0a9 Merge pull request #1 from pablowmartinez/dev-a
9470408 Implementación de las operaciones de suma y resta
2d395c3 Estructura inicial del proyecto
d731128 Initial commit
```

Este es el contenido del archivo funciones.h:

```
funciones.h X
1  #ifndef FUNCIONES_H_INCLUDED
2  #define FUNCIONES_H_INCLUDED
3
4  #include <math.h>
5
6  float potencia(float base, float exponente);
7  float raiz(float numero, float indice);
8  float multiplicar(float a, float b);
9  float dividir(float a, float b);
10 float suma(float a, float b);
11 float resta(float a, float b);
12
13 #endif // FUNCIONES_H_INCLUDED
14
```

Ahora nos moveremos al commit inicial con el comando:

- `git checkout 2d395c3`

Observar en la siguiente el cambio desde main a el commit y la advertencia de Git:



UNLaM

Dto. Ingeniería e Investigaciones Tecnológicas

```

MINGW64 ~/Documents/UNLaM/2025-C2/ejemplo_git_github/dev_c/calculadora-c (main)
$ git checkout 2d395c3
Note: switching to '2d395c3'.

You are in 'detached HEAD' state. You can look around, make experimental
changes and commit them, and you can discard any commits you make in this
state without impacting any branches by switching back to a branch.

If you want to create a new branch to retain commits you create, you may
do so (now or later) by using -c with the switch command. Example:

    git switch -c <new-branch-name>

Or undo this operation with:

    git switch -

Turn off this advice by setting config variable advice.detachedHead to false

HEAD is now at 2d395c3 Estructura inicial del proyecto
MINGW64 ~/Documents/UNLaM/2025-C2/ejemplo_git_github/dev_c/calculadora-c ((2d395c3...))
$

```

Ahora, observemos el contenido del archivo funciones.h:

```

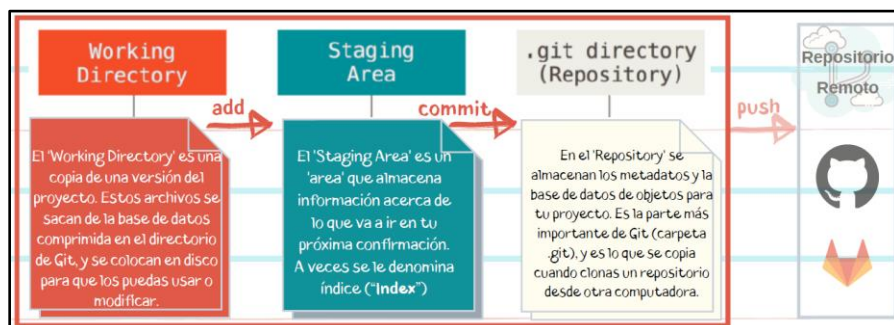
funciones.h x
1  #ifndef FUNCIONES_H_INCLUDED
2  #define FUNCIONES_H_INCLUDED
3
4
5
6  #endif // FUNCIONES_H_INCLUDED
7

```

Podemos ver que efectivamente es el contenido inicial (archivo vacío).

ESCENARIO 2: Queremos que nuestra rama “vuelva atrás” y descarte los commits posteriores.

Para entender esto, es necesario recordar que eran el Working Directory, Staging Area y el Repository:





Comando:

- `git reset`

Sin embargo, es crucial elegir correctamente el modo, según queramos conservar o borrar los cambios del área de trabajo (Working Directory).

Opción A: Borrar commits y también cambios del working directory.

Comando: `git reset --hard <commit_id>`

- Elimina los commits posteriores a `<commit_id>`.
- El código queda exactamente como estaba en ese commit.
- Peligroso si ya se había hecho push, porque se perderá el historial remoto.

Opción B: Borrar commits pero conservar los archivos modificados.

Comando: `git reset --soft <commit_id>`

- Los cambios quedan en “staging area”.
- Ideal si se requiere rehacer los commits.

Opción C: Borrar commits y mantener cambios en el directorio (sin staged)

Comando: `git reset --mixed <commit_id>`

- Los archivos modificados siguen en la carpeta de trabajo, pero no están en staging.

Tarea: aplicar las tres opciones en el proyecto y observar lo que sucede en cada caso.

ESCENARIO 3: Ya se ha realizado un push al repositorio remoto

Si el commit ya está en GitHub y otros colaboradores podrían haberlo descargado, **NO** se debe utilizar `reset --hard` debido a que se rompería el historial.

En ese caso, usa `git revert`, que crea un commit nuevo que “deshace” los cambios. El comando que se debe utilizar es:

```
git revert <commit_id>
```

De esta manera se **preserva la historia** y es **la forma segura** de deshacer un cambio público.



UNLaM

Dto. Ingeniería e Investigaciones Tecnológicas

Resumen rápido

Situación	Comando	Qué hace
Probar un commit anterior	<code>git checkout <id></code>	Solo moverse temporalmente
Volver atrás sin perder cambios	<code>git reset --soft <id></code>	Mantiene los archivos en staging
Volver atrás conservando los archivos sin staging	<code>git reset --mixed <id></code>	Mantiene los archivos modificados
Volver atrás borrando todo	<code>git reset --hard <id></code>	Elimina commits y cambios
Deshacer un commit ya subido a remoto	<code>git revert <id></code>	Crea un commit "opuesto"